

Advanced Python Programming For Data Science

Chapter 1: Advanced Python Concepts and Best Practices

Chapter Overview

This chapter focuses on:

- Writing professional, scalable Python code
- Using advanced language features
- Preparing students for real-world data science and engineering workflows

Duration: 7 hours

Marks Weightage: 9 Marks

Learning Outcomes

After completing this chapter, students will be able to:

- Use advanced Python data structures efficiently
- Apply iterators, generators, and decorators
- Handle flexible function arguments
- Use lambda expressions appropriately
- Debug programs systematically
- Implement logging for production systems
- Organize code using modules and packages

Chapter Structure

1. Review of Python essentials and coding conventions
2. Advanced data structures
3. Functions and lambda expressions
4. Object-Oriented Programming for data science
5. Debugging and logging
6. Working with modules and packages

1.1 Review of Python Essentials and Coding Conventions

Python Essentials: Already covered in “Python for Beginners” Slides

Topics assumed as prior knowledge:

- Python syntax and indentation
- Variables and data types
- Conditional and looping constructs
- Basic functions and returns
- Built-in data structures
- PEP8 coding conventions (Will cover this later)

1.2 Advanced Data Structures

Why Advanced Data Structures?

Advanced data structures help:

- Improve performance
- Reduce memory usage
- Enable lazy evaluation
- Write cleaner and reusable code

Covered in this section:

- Collections
- Iterators
- Generators
- Decorators

1.2 Advanced Data Structures — The collections Module

- The collections module provides **specialized container data types**
- These are optimized alternatives to Python's built-in:
 - dict, list, set, tuple
- Helps in writing:
 - Cleaner code
 - More efficient data handling
 - Safer logic for real-world applications
- Widely used in data science, analytics, and backend systems

Counter — Frequency Counting

Description

- Counter is a dictionary subclass for counting **hashable objects**
- Automatically maintains frequency counts
- Simplifies tallying operations

Use Cases

- Word frequency analysis
- Category counts
- Event tracking

python

```
from collections import Counter
```

```
Counter(['a', 'b', 'a'])
```

defaultdict — Automatic Default Values

Description

- defaultdict automatically assigns a default value to missing keys
- Avoids explicit key existence checks
- Prevents KeyError

Use Cases

- Grouping data
- Counting occurrences
- Accumulating values

```
from collections import defaultdict
```

```
d = defaultdict(int)
```

```
d['count'] += 1
```

namedtuple — Lightweight Objects

Description

- namedtuple creates tuple-like objects with named attributes
- Combines readability with memory efficiency
- Objects are immutable

Use Cases

- Representing structured records
- Clean data modeling
- Read-only configurations

```
1 from collections import namedtuple
2
3 Point = namedtuple('Point', ['x', 'y'])
4 p = Point(10, 20)
5
```

deque — Fast Queue Operations

Description

- deque is a double-ended queue
- Optimized for fast insertions and deletions from both ends
- More efficient than lists for queue-like behavior

Use Cases

- Queues and stacks
- Sliding window problems
- Task scheduling

```
1 from collections import deque
2
3 q = deque([1, 2])
4 q.appendleft(0)
5 q.append(5)
```



Summary — collections Module

- Counter → Frequency counting
- defaultdict → Safe dictionary operations
- namedtuple → Structured, readable data
- deque → Efficient queue and stack handling

1.2 Advanced Data Structures — Iterators

What are Iterators?

- An **iterator** is an object that produces values **one at a time**
- It maintains internal state between iterations
- Iteration stops automatically when values are exhausted
- Enables **memory-efficient traversal** of data

How Iterators Work?

- **Iterators implement:**
 - `__iter__()` → returns the iterator object
 - `__next__()` → returns the next value
- When no values remain:
 - `StopIteration` is raised internally
- Values are produced one by one
- Useful when processing data sequentially

```
1 data = [1, 2, 3]
2 it = iter(data)
3 print(next(it))
4 print(next(it))
5 print(next(it))
6 print(next(it))
7
```

Where Iterators Are Used?

- File reading (line by line)
- Database cursors
- Streaming data sources
- Large datasets that cannot fit into memory

Advanced Data Structures — Generators

What are Generators?

- A **generator** is a special type of iterator
- Defined using the **yield** keyword
- Produces values lazily, on demand
- Pauses execution and resumes later

Why Generators Matter?

- Reduce memory usage
- Improve performance for large data
- Ideal for data pipelines and ETL processes
- Enable **lazy evaluation**

Example — Generator

- Values are generated **only when requested**
- Execution resumes from the last yield

```
1 # Standard Function (High Memory)
2 def get_squares(n):
3     return [i**2 for i in range(n)] # Creates full list in memory
4
5 # Generator (Low Memory)
6 def get_squares_gen(n):
7     for i in range(n):
8         yield i**2 # Yields one value at a time
9
10 gen = get_squares_gen(1000000)
11 print(next(gen)) # 0
12 print(next(gen)) # 1
```

Generator Use Cases

- Reading large files
- Streaming sensor or log data
- Batch processing in data science
- Implementing pipelines step by step

Iterator vs Generator (Conceptual)

Iterator

- Manually implemented
- More verbose

Generator

- Simpler syntax
- Automatically handles iteration state

Python Decorators

What is a decorator?

- A **decorator** is a function that:
 - Takes another function as input
 - Extends or modifies its behavior
- Does not change the original function's code
- It follows the principle of **separation of concerns**.

Why Decorators Are Useful

- Add functionality transparently
- Reduce code duplication
- Improve code organization
- Widely used in frameworks and libraries

Example — Decorator

- Wraps existing function behavior
- Enhances functionality without modification

```
1 def my_decorator(func):  
2     def wrapper():  
3         print("Before function call")  
4         func()  
5         print("After function call")  
6     return wrapper
```

Applying a Decorator

- @ syntax makes decorators readable
- Common in professional Python codebases

```
1 @my_decorator
2 def say_hello():
3     print("Hello")
```

Common Decorator Use Cases

- Logging
- Performance timing
- Authentication and authorization
- Validation and access control
- Caching results

```
def my_logger(func):  
    def wrapper():  
        print("Log: Function execution started.")  
        func()  
        print("Log: Function execution finished.")  
    return wrapper  
  
@my_logger  
def process_data():  
    print("Processing data...")  
  
process_data()
```



Summary — Iterators, Generators, Decorators

- Iterators → Controlled, sequential data access
- Generators → Lazy, memory-efficient iteration
- Decorators → Clean and reusable behavior enhancement

These concepts are core to advanced Python and data engineering.

1.3 Functions and Lambda Expressions

Basic function concepts were covered earlier like:

- Function definition
- Return values
- Default arguments
- Function calls

Will focus : Advanced Argument Handling and Lambdas.

1.3 Functions: Positional & Keyword Arguments

Why *args and **kwargs

- In real applications:
 - Number of arguments may vary
 - APIs must be flexible
 - Wrapper and decorator functions need extensibility

Understanding *args

- ***args** allows variable **positional arguments**
- Internally stored as a **tuple**
- Name **args** is a convention

Common use cases:

- Handles dynamic input size
- Keeps function signatures clean
- Improves reusability
- Aligns with real-world data variability

```
1 def average(*values):  
2     return sum(values) / len(values)  
3  
4 average(1)  
5  
6 average(1, 2, 3)
```

Understanding ****kwargs**

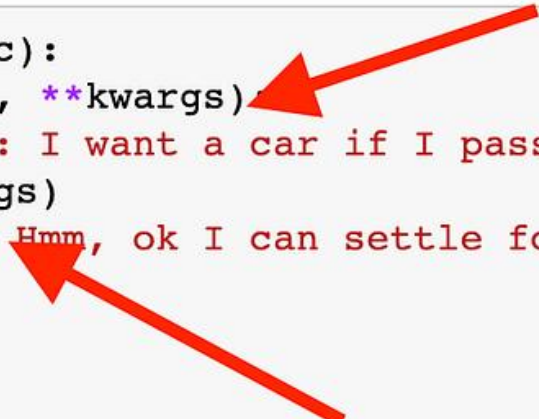
- ****kwargs** allows variable **keyword arguments**
- Internally stored as a **dictionary**
- Keys are argument names

Common use cases:

- Supports extensibility without breaking existing code
- Improves readability through named arguments
- Allows selective parameter passing
- Aligns with real-world application requirements

```
1 def train_model(model_name, **hyperparameters):  
2     print(f"Training {model_name}...")  
3     for key, value in hyperparameters.items():  
4         print(f"Setting {key} to {value}")  
5  
6 train_model("RandomForest", trees=100, max_depth=5, learning_rate=0.01)
```

```
] def function_decorator(func):  
    def wrapped_func(*args, **kwargs):  
        print('Before test: I want a car if I pass the exam with flying colors')  
        func(*args, **kwargs)  
        print('After test: Hmm, ok I can settle for a bicycle because I passed')  
    return wrapped_func  
  
@function_decorator  
def test(marks):  
    print(f'I passed the exam but with average grades. My marks were {marks}')
```



```
] test(50)
```

```
Before test: I want a car if I pass the exam with flying colors  
I passed the exam but with average grades. My marks were 50  
After test: Hmm, ok I can settle for a bicycle because I passed
```

Order of Function Parameters

Python function parameters must follow this order:

1. Positional arguments
2. `*args` (variable positional arguments)
3. Default (keyword) arguments
4. `**kwargs` (variable keyword arguments)

This order applies to all Python functions, regardless of complexity.

```
1 def func_1(a=10, b):  
2     pass  
3  
4 def func_2(name, **details, *scores):  
5     pass  
6  
7 def func_3(name, *args, mode="Standard", **kwargs):  
8     pass  
9  
10 def func_2(name=12, *scores):  
11     pass
```

1.3 Lambda Expressions

A lambda expression:

- Is an anonymous function
- Written in a single line
- Used for small, simple logic
- Syntax:

lambda arguments: expression

```
1 data = [(1, 'B'), (2, 'A'), (3, 'C')]
2
3 # Sort by the second element (the letter)
4 # Instead of defining a full function, we use lambda
5 data.sort(key=lambda x: x[1])
6
7 print(data)
8 # Output: [(2, 'A'), (1, 'B'), (3, 'C')]
```

The Quick-Tool: Lambda Expressions

The Blueprint (What it is)

A small, anonymous function defined with the ``lambda`` keyword. It can take any number of arguments but can only have one expression.

The Use Case (Why you need it)

For short, disposable functions where a full ``def`` statement is overkill. Extremely common in data analysis with libraries like Pandas.

- Sorting a list of complex objects by a specific attribute.
- ⊕ Using ``map()``, ``filter()``, or ``reduce()``.
- ⊕ Applying a simple transformation to a Pandas Series or DataFrame column.

The Implementation (How it works)

Standard Function (Verbose)

```
def get_year(date_string):  
    return date_string.split('-')[0]  
  
df['year'] = df['date_column'].apply(get_year)
```

Lambda Function (Concise & Pythonic)

```
df['year'] = df['date_column'].apply(lambda x:  
    x.split('-')[0])
```

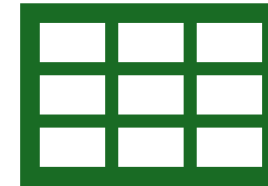
Use lambdas for simplicity and readability when the logic is a single expression. If it gets more complex, define a proper function.

1.4 Object-Oriented Programming for Data Science



Previously covered:

- Classes and objects
- Encapsulation
- Inheritance
- Polymorphism



OOP will be used later for:

- Pipelines
- Models
- Reusable components



1.5 Exception handling, debugging, and logging

Exception handling was covered in the Python for Beginners class; this chapter focuses on debugging and logging.

1.5 Debugging in Python

Debugging is the process of:

- Identifying bugs
- Understanding program flow
- Fixing logical and runtime errors

Techniques:

- Print-based debugging
- Stack trace analysis
- Debuggers (pdb, IDE tools)

Both `pdb.set_trace()` and `breakpoint()` pause execution for debugging, but `breakpoint()` is a higher-level, configurable debugging hook introduced in Python 3.7, while `pdb.set_trace()` directly invokes the `pdb` debugger.

```
1 def calculate_average(numbers):
2     import pdb; pdb.set_trace()
3     total = sum(numbers)
4     avg = total / len(numbers)
5     return avg
6
7 calculate_average([10, 20, 30])
```

PROBLEMS TERMINAL DEBUG CONSOLE

Microsoft Windows [Version 10.0.26100.7462]
(c) Microsoft Corporation. All rights reserved.
The value specified in an AutoRun registry key could not be parsed.

C:\instructor\AdvancedPythonProgramming>C:\instructor\AdvancedPythonProgramming
The value specified in an AutoRun registry key could not be parsed.

(venv) C:\instructor\AdvancedPythonProgramming>python test.py
> c:\instructor\advancedpythonprogramming\test.py(3)calculate_average()
-> total = sum(numbers)
(Pdb) n
> c:\instructor\advancedpythonprogramming\test.py(4)calculate_average()
-> avg = total / len(numbers)
(Pdb) total
60
(Pdb) █

Example: Debugging

1.5 Logging in Python

The logging Module: Essential for tracking events that happen when some software runs, especially in automated pipelines.

In simpler term, Logging:

- Records application events
- Replaces excessive print()
- Essential for production systems

Logging Levels (Hierarchy)

1. **DEBUG:** Detailed information, typically of interest only when diagnosing problems.
2. **INFO:** Confirmation that things are working as expected.
3. **WARNING:** An indication that something unexpected happened (e.g., "Disk space low").
4. **ERROR:** Due to a more serious problem, the software has not been able to perform some function.
5. **CRITICAL:** A serious error, indicating that the program itself may be unable to continue running.

Example - Logging

```
1 import logging
2
3 logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %
    (message)s')
4
5 def connect_db():
6     logging.info("Attempting database connection...")
7     # connection logic
8     logging.error("Connection failed!")
9
10 connect_db()
11
12 # Output:
13 # 2025-12-23 01:42:43,860 - INFO - Attempting database connection...
14 # 2025-12-23 01:42:43,860 - ERROR - Connection failed!
15
```

Differences between Debugging and Logging

Aspect	Debugging	Logging
Purpose	Identify and fix errors in code	Record program events and behavior
Usage phase	Development and testing	Development and production
Program execution	Pauses execution	Runs continuously
Interaction	Interactive (step-by-step inspection)	Non-interactive
Typical tools	<code>print()</code> , <code>pdb</code> , <code>breakpoint()</code>	logging module
Output	Temporary, not stored	Persistent (files, console, systems)
Production suitability	Not suitable	Essential
Primary goal	Understand and fix bugs	Monitor, audit, and diagnose issues over time

1.6 Working with Modules and Packages

Python Modules

A module:

- Is a single .py file
- Contains reusable code

Benefits:

- Code reuse
- Better organization
- Easier testing

Python Packages

A package:

- Is a directory of modules
- Enables large project structures
- Uses `__init__.py`: A special file (often empty) that tells Python that the directory should be treated as a package.

In short: A way of structuring Python's module namespace by using "dotted module names".

Importing Strategies

1. `import my_module` (Namespaces execution)
2. `from my_module import my_function` (Direct access)
3. `from my_package.subpackage import module`

Best Practice: Avoid `from module import *` (wildcard import) as it pollutes the namespace and makes code harder to debug.

Blueprints for Scale: Modules and Packages



Module

- **Blueprint:** A single file (`my_helpers.py`).
- **What it is:** A collection of related functions, classes, and variables.
- **How to use:** `import my_helpers` or `from my_helpers import specific_function`.



Package

- **Blueprint:** A directory containing an `__init__.py` file and other modules.
- **What it is:** A collection of related modules organized in a directory hierarchy. The `__init__.py` file tells Python to treat the directory as a package. It can be empty or can run initialization code.
- **How to use:** `import data_pipeline.ingestion` or `from data_pipeline.transformation import clean_data`.

```
my_project/  
├── main.py  
└── data_pipeline/  
    ├── __init__.py  
    ├── ingestion.py      # Module for data loading  
    ├── transformation.py # Module for cleaning/feature engineering  
    └── validation.py     # Module for data quality checks
```

Use modules for simple collections of helpers. Use packages to structure complex projects, separate concerns, and build reusable libraries.

Why Modules and Packages Matter

- Team collaboration
- Clean architecture
- Scalable applications
- Maintainable data science pipelines



Chapter 1 — Summary: Advanced Python Concepts and Best Practices

- Reviewed Python essentials and coding conventions (assumed prior knowledge)
- Covered advanced data structures: collections, iterators, generators, decorators
- Explored advanced function usage: positional vs keyword arguments, `*args`, `**kwargs`, lambda expressions
- Revisited Object-Oriented Programming for data science (previously covered)
- Learned debugging techniques using `print`, `pdb`, and `breakpoint()`
- Introduced logging concepts for production-ready code
- Understood modules and packages for scalable Python projects